

AIM2 Preparation

Week 1 - Day 5: Loops & Data Processing

5-Minute Review Notes | for, while, range, enumerate, zip & Comprehensions

Day 5 Goal

Learn how to repeat operations efficiently, process collections of data, track indexes, combine sequences, create transformed lists, and work with nested structures.

1. Why Loops?

Loops repeat a block of code. They are useful when the same operation must be performed for many values or records.

```
for number in range(1, 6):  
    print(number)
```

2. for Loop

```
students = ["Krishna", "Rahul", "Priya"]  
  
for student in students:  
    print(student)
```

A for loop processes one item from an iterable at a time.

3. range()

```
for number in range(1, 21):  
    print(number)
```

Syntax	Meaning
range(stop)	0 up to stop-1
range(start, stop)	start up to stop-1
range(start, stop, step)	same range with a custom step

Remember: the stop value is excluded.

4. while Loop

```
count = 1  
  
while count <= 5:  
    print(count)  
    count += 1
```

A while loop continues as long as its condition remains True. The condition must eventually change or the loop can run forever.

5. for vs while

for	while
Best when iterating over a collection/range	Best when repetition depends on a condition
Usually has a natural stopping point	You must ensure the condition becomes False

6. enumerate()

```
students = ["Krishna", "Rahul", "Priya"]

for index, student in enumerate(students):
    print(index, student)
```

enumerate() gives both the index and the value while looping.

7. zip()

```
names = ["Krishna", "Rahul", "Priya"]
scores = [85, 75, 91]

for name, score in zip(names, scores):
    print(name, score)
```

zip() pairs corresponding items from multiple iterables, which is useful when related data is stored in separate lists.

8. List Comprehensions

```
numbers = [1, 2, 3, 4, 5]
squares = [number ** 2 for number in numbers]
```

A list comprehension is a compact way to create a new list by transforming items from another iterable.

```
even_numbers = [number for number in range(1, 11) if number % 2 == 0]
```

A condition can also be included to filter values.

9. Nested Loops

```
for row in range(1, 4):
    for column in range(1, 4):
        print(row, column)
```

A nested loop is a loop inside another loop. For every iteration of the outer loop, the inner loop runs through its iterations.

10. Processing Student Data

```
students = [
    {"Name": "Krishna", "Math": 85, "Python": 92},
    {"Name": "Rahul", "Math": 75, "Python": 80}
]

for student in students:
    total = student["Math"] + student["Python"]
    average = total / 2
    print(student["Name"], average)
```

This combines lists, dictionaries, loops, and calculations to process many records automatically.

11. Useful Loop Thinking

- What collection or range am I repeating over?
- What should happen during each iteration?
- Do I need the value only, or both index and value?
- Are two lists supposed to be processed together?
- Does the loop have a guaranteed stopping condition?
- Can a simple transformation be expressed clearly with a list comprehension?

Day 5 Practice Highlights

- Used for with range(1, 21).

- Practiced while loops and updating the loop variable.
- Used enumerate() to retrieve indexes and values.
- Used zip() to combine related sequences.
- Created lists using list comprehensions.
- Practiced conditional list comprehensions.
- Worked with nested loops.
- Applied loops to basic data-processing tasks.

Quick Syntax Cheat Sheet

```
for item in iterable:
    ...

for n in range(start, stop, step):
    ...

while condition:
    ...

for index, value in enumerate(items):
    ...

for a, b in zip(list1, list2):
    ...

new_list = [expression for item in iterable]
filtered = [item for item in iterable if condition]
```

Key Takeaways

- for loops are ideal for iterating over collections and ranges.
- while loops repeat while a condition is True.
- range() generates a sequence of integers and excludes the stop value.
- enumerate() provides index + value.
- zip() processes corresponding values from multiple iterables together.
- List comprehensions create transformed or filtered lists concisely.
- Nested loops handle repeated work across multiple levels.
- Loops are essential for automating data processing before moving into vectorized NumPy/Pandas operations.

Progress

Week 1 Day 5 - Loops & Data Processing: COMPLETE

Practice focus: for, while, range(), enumerate(), zip(), list comprehensions, and nested loops.